

Class 6: R Functions Lab

Joshua Robinson (PID: A18574862)

Table of contents

Background	1
A first function	1
Q2. Write a <code>generate_dna()</code> function	3
Write a <code>generate_protein()</code> function	5
Generate random protein sequences of length 6 to 13	6
Are our peptides “unique in nature”?	7
Connecting our findings to immunology	8

Background

All functions in R have at least 3 things:

- a *name* (We pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either “required” or “optional”. The later will have fall-back default values that will be used if the user does not specify them.

```
add <- function(x,y=0)
{x + y}
```

Can we use our new function:

```
add(10,100)
```

```
[1] 110
```

Q1b: For your second version, adopt your first function so it can take a single input vector or two inputs as before. For example add(4,7) abd add c(4,7,10). [1 pt]

```
add <- function(x,y=0)
{x + y}
```

```
add <- function(x,y=0)
sum(x + y)
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c. To do on your own... create a third version of your function that can add any number of inputs that the user provides. For example, add(1,2,3,-4)

```
add <- function(x,y=0, z=0)
{sum(x,y,z)}
```

```
add <- function(...) {
  sum(...)
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

```
ans <- add(x=1,y=2,z=13)
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

```
add <- function(x,y=0, z=0)
{return(sum(x,y,z))
  cat("Is it break time yet?\n")}
add(10,100)
```

```
[1] 110
```

Q2. Write a generate_dna() function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size=3)
```

```
[1] 4 3 1
```

```
sample(1:5, size=60, replace = TRUE)
```

```
[1] 5 3 4 3 3 4 5 2 5 5 3 3 1 4 2 1 3 3 1 4 2 1 3 5 2 3 4 3 3 1 3 4 5 5 3 5 3 2
[39] 1 3 1 2 2 1 2 2 2 1 4 2 1 2 5 1 2 3 2 2 2 4
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(x=c("A","C","G","T"), size=10, replace = TRUE)
```

```
[1] "C" "G" "C" "C" "G" "G" "T" "A" "A" "G"
```

Q2: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len=10) {
  sample(x=c("A","C","G","T"), size=len, replace = TRUE)
}
```

```
generate_dna (len=10)
```

```
[1] "G" "A" "T" "C" "G" "T" "G" "A" "G" "G"
```

```
generate_dna()
```

```
[1] "T" "T" "A" "C" "A" "G" "A" "A" "G" "A"
```

```
generate_dna(len=100)
```

```
[1] "G" "G" "A" "T" "T" "T" "G" "T" "A" "A" "G" "A" "G" "T" "T" "G" "C" "G"
[19] "C" "A" "A" "A" "A" "G" "G" "A" "G" "A" "A" "T" "G" "T" "C" "A" "T" "C"
[37] "A" "A" "A" "G" "T" "T" "A" "G" "T" "A" "C" "T" "A" "T" "C" "G" "T" "A"
[55] "A" "C" "A" "A" "C" "C" "G" "G" "T" "T" "T" "A" "T" "C" "T" "A" "T" "A"
[73] "T" "C" "T" "G" "T" "G" "C" "C" "A" "G" "G" "C" "A" "T" "A" "T" "G" "A"
[91] "C" "A" "T" "C" "C" "T" "T" "G" "T" "T"
```

Q2b: Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

```
generate_dna <- function(len=10, single.element = TRUE) {
  ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)
  #cat("Hello...")
  if(single.element) {
    #cat("is it me you are looking for...")
    ans <- paste(ans,collapse = "")
  }
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()` and `return()`

```
generate_dna(single.element = TRUE)
```

```
[1] "CACCATGGGG"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
> len9
CGAAGTC
```

```
cat("hello\n there")
```

```
hello
  there
```

```
generate_dna <- function(len=10, single.element = TRUE) {
  ans <- sample(x=c("A","C","G","T"), size=len, replace = TRUE)
  if(single.element) {
    ans <- paste(ans,collapse = "")}
  ## Format as FASTA with an ID line
  cat(paste(">len", len, "\n", sep=""))
  cat(ans)
  cat("\n")
  ##
}
```

```
generate_dna(44)
```

```
>len44
TGTTACATCCATGGTGTTCGGCCCTTGTAATAAGTGACTCCTA
```

Write a generate_protein() function

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```
generate_protein <- function(len=10) {
  aa <- c("A", "R", "N", "D", "C", "Q", "E", "G", "H", "I",
         "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")
  prot <- sample(x=aa, size=len, replace = TRUE)
  prot <- paste(prot, collapse="")
  return(prot)
}
```

```
generate_protein(6)
```

```
[1] "QCVPLI"
```

Generate random protein sequences of length 6 to 13

Q4. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for(l in 6:13) {  
  cat(">", l, "\n", sep="")  
  cat(generate_protein(l), "\n")  
}
```

```
>6  
PHSCPQ  
>7  
CYSAYLS  
>8  
LSSVEEHG  
>9  
NWSSTEDCH  
>10  
CPHGSVGYGC  
>11  
EKVHWYRYWCF  
>12  
DQQPYCKKNGTE  
>13  
WSNNIPVNQYYDS
```

```
generate_protein(6)
```

```
[1] "EQNFGV"
```

```
generate_protein(7)
```

```
[1] "VHYAFLM"
```

```
generate_protein(8)
```

```
[1] "CNEWECGY"
```

```
generate_protein(9)
```

```
[1] "DVNMMYANE"
```

Are our peptides “unique in nature”?

Q5. BLASTp search against nr — are your peptides “unique in nature”? Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

length	Ide	Cov	Unique
6	100	100	N
7	100	100	N
8	100	88	Y
9	100	89	Y
10	90	100	Y
11	90	91	Y
12	77	100	Y
13	100	69	Y

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

Peptides start to look unique at length 8, where 100% identity and coverage are no longer both observed.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^n for a peptide of length n) and to the size of the known protein universe. [1 pt]

Very short peptides are almost always found in nr because the sequence space is small relative to the known protein database, while longer peptides have a larger sequence space that exceeds the size of known proteins, making exact matches unlikely.

Connecting our findings to immunology

Q6. In 3-6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

The minimum length for uniqueness is around 8-10 amino acids, where the sequence space becomes large enough to reduce random matches in known proteins. This suggests that very short peptides are not reliably unique identifiers of specific proteins. Therefore, it would be a poor design for the immune system to present very short peptides because they could match many different proteins, increasing the risk of non-specific or self-reactive immune responses.